Lab Experiments 3-12

Data Handling & Visualization: Python + R, Code & Output

Each experiment: Python code + output, followed by R code + output

Date: 17/07/2026

Experiment 3

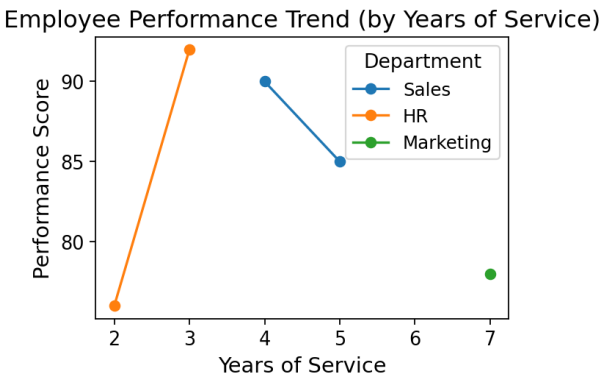
Employee Performance Evaluation

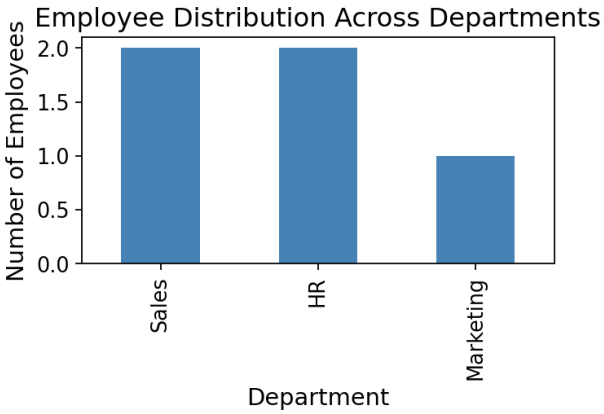
PYTHON

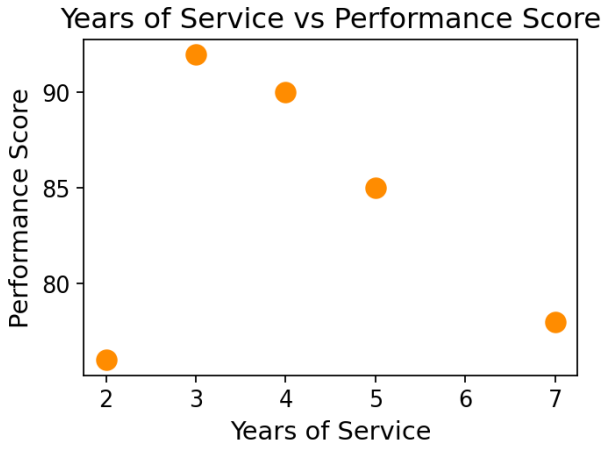
Code

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.DataFrame({
    'EmployeeID': [1, 2, 3, 4, 5],
    'Department': ['Sales', 'HR', 'Marketing', 'Sales', 'HR'],
    'YearsOfService': [5, 3, 7, 4, 2],
    'PerformanceScore': [85, 92, 78, 90, 76]
})

# 1. Line chart - performance trend over time
# No time/date column is given, so Years of Service is used as the time-progression
# proxy (x-axis), with each department as its own line/legend entry.
plt.figure(figsize=(7, 5))
for dept in df['Department'].unique():
    sub = df[df['Department'] == dept].sort_values('YearsOfService')
    plt.plot(sub['YearsOfService'], sub['PerformanceScore'], marker='o', label=dept)
plt.title("Employee Performance Trend (by Years of Service)")
plt.xlabel("Years of Service"); plt.ylabel("Performance Score")
plt.legend(title="Department")
plt.tight_layout()
plt.show()

# 2. Bar chart - employee distribution across departments
dept_counts = df['Department'].value_counts()
plt.figure(figsize=(6, 5))
dept_counts.plot(kind='bar', color='steelblue')
plt.title("Employee Distribution Across Departments")
plt.xlabel("Department"); plt.ylabel("Number of Employees")
plt.tight_layout()
plt.show()

# 3. Scatter plot - years of service vs performance score
plt.figure(figsize=(7, 5))
plt.scatter(df['YearsOfService'], df['PerformanceScore'], s=100, color='darkorange')
plt.title("Years of Service vs Performance Score")
plt.xlabel("Years of Service"); plt.ylabel("Performance Score")
plt.tight_layout()
plt.show()
# Insight: no strong linear correlation across this small sample - HR's 3-year
# employee outperforms Marketing's 7-year employee, suggesting performance here
# is not simply tenure-driven.
```

Output





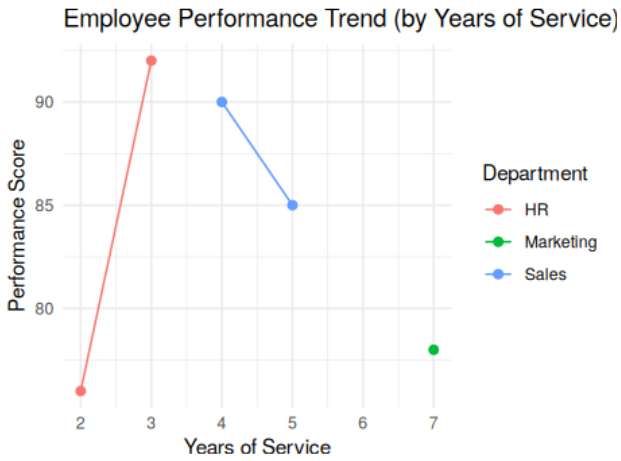


R

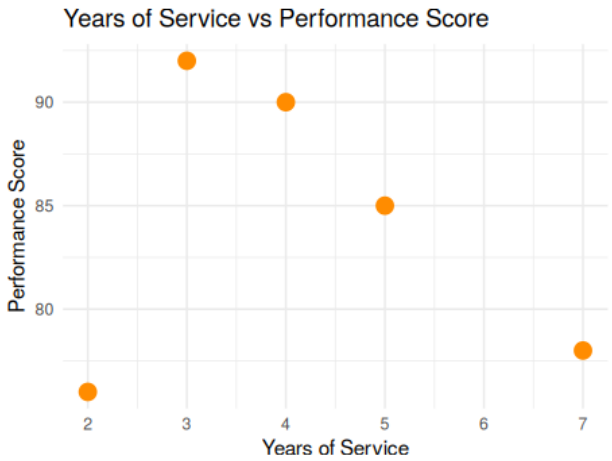
Code

```
library(ggplot2)

df &lt;- data.frame(
  EmployeeID = 1:5,
  Department = c('Sales','HR','Marketing','Sales','HR'),
  YearsOfService = c(5,3,7,4,2),
  PerformanceScore = c(85,92,78,90,76)
)

# 1. Line chart - performance trend over time
# No time/date column is given, so Years of Service is used as the time-progression
# proxy (x-axis), with each department as its own line/legend entry.
p1 &lt;- ggplot(df, aes(x=YearsOfService, y=PerformanceScore, color=Department)) +
  geom_line() + geom_point(size=2) +
  labs(title="Employee Performance Trend (by Years of Service)",
       x="Years of Service", y="Performance Score") +
  theme_minimal()
print(p1)

# 2. Bar chart - employee distribution across departments
dept_counts &lt;- as.data.frame(table(df$Department))
colnames(dept_counts) &lt;- c('Department','Count')
p2 &lt;- ggplot(dept_counts, aes(x=Department, y=Count)) +
  geom_col(fill="steelblue") +
  labs(title="Employee Distribution Across Departments", y="Number of Employees") +
  theme_minimal()
print(p2)

# 3. Scatter plot - years of service vs performance score
p3 &lt;- ggplot(df, aes(x=YearsOfService, y=PerformanceScore)) +
  geom_point(size=4, color="darkorange") +
  labs(title="Years of Service vs Performance Score",
       x="Years of Service", y="Performance Score") +
  theme_minimal()
print(p3)
# Insight: no strong linear correlation across this small sample - HR's 3-year
# employee outperforms Marketing's 7-year employee, suggesting performance here
# is not simply tenure-driven.
```

Output


Employee Performance Trend (by Years of Service)
Performance Score
90
85
80
2
3
4
5
6
7
Years of Service
Department
HR
Marketing
Sales


Employee Distribution Across Departments
Number of Employees
2.0
1.5
1.0
0.5
0.0
HR
Marketing
Sales
Department


Years of Service vs Performance Score
Performance Score
90
85
80
2
3
4
5
6
7
Years of Service

Experiment 4

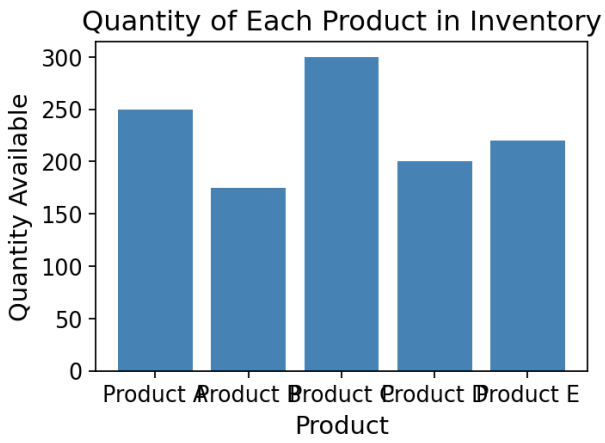
Product Inventory Management

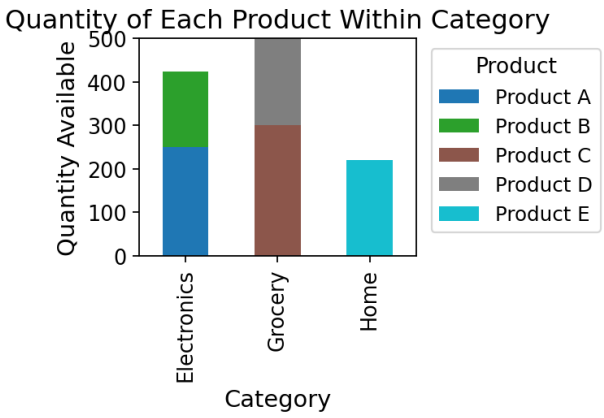
PYTHON

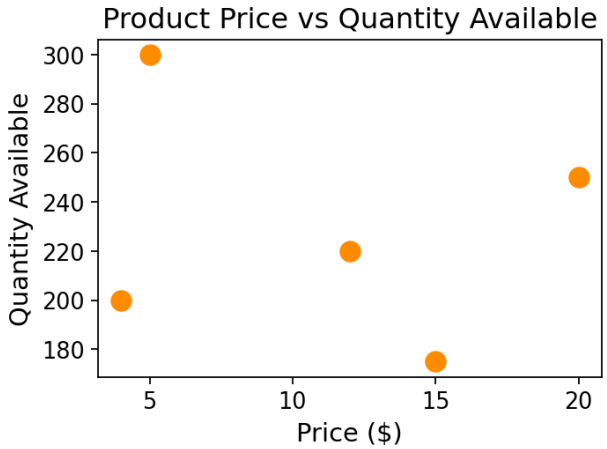
Code

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.DataFrame({
    'ProductID': [1, 2, 3, 4, 5],
    'ProductName': ['Product A', 'Product B', 'Product C', 'Product D', 'Product E'],
    'QuantityAvailable': [250, 175, 300, 200, 220]
})

# Category and Price columns are not in the given table, so they are assumed here
# to satisfy Task 2 (stacked bar by category) and Task 3 (price vs quantity).
df['Category'] = ['Electronics', 'Electronics', 'Grocery', 'Grocery', 'Home']
df['Price'] = [20, 15, 5, 4, 12]

# 1. Bar chart - quantity of each product
plt.figure(figsize=(7, 5))
plt.bar(df['ProductName'], df['QuantityAvailable'], color='steelblue')
plt.title("Quantity of Each Product in Inventory")
plt.xlabel("Product"); plt.ylabel("Quantity Available")
plt.tight_layout()
plt.show()

# 2. Stacked bar chart - quantity by product within category
pivot = df.pivot_table(index='Category', columns='ProductName', values='QuantityAvailable', fill_value=0)
pivot.plot(kind='bar', stacked=True, figsize=(7, 5), colormap='tab10')
plt.title("Quantity of Each Product Within Category")
plt.xlabel("Category"); plt.ylabel("Quantity Available")
plt.legend(title="Product", bbox_to_anchor=(1.02, 1), loc='upper left')
plt.tight_layout()
plt.show()

# 3. Scatter plot - price vs quantity available
plt.figure(figsize=(7, 5))
plt.scatter(df['Price'], df['QuantityAvailable'], s=100, color='darkorange')
plt.title("Product Price vs Quantity Available")
plt.xlabel("Price ($)"); plt.ylabel("Quantity Available")
plt.tight_layout()
plt.show()
# Insight: lower-priced products (Grocery) tend to be stocked in similar or
# slightly higher quantities than pricier Electronics items.
```

Output







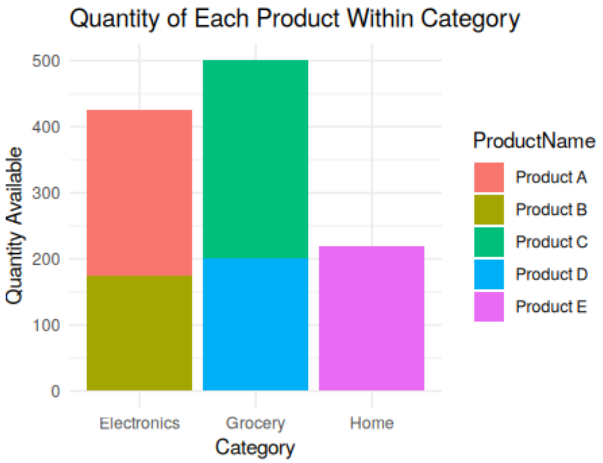
R

Code

```
library(ggplot2)

df &lt;- data.frame(
  ProductID = 1:5,
  ProductName = c('Product A','Product B','Product C','Product D','Product E'),
  QuantityAvailable = c(250,175,300,200,220)
)

# Category and Price columns are not in the given table, so they are assumed here
# to satisfy Task 2 (stacked bar by category) and Task 3 (price vs quantity).
df$Category &lt;- c('Electronics','Electronics','Grocery','Grocery','Home')
df$Price &lt;- c(20,15,5,4,12)

# 1. Bar chart - quantity of each product
p1 &lt;- ggplot(df, aes(x=ProductName, y=QuantityAvailable)) +
  geom_col(fill="steelblue") +
  labs(title="Quantity of Each Product in Inventory", x="Product", y="Quantity Available") +
  theme_minimal()
print(p1)

# 2. Stacked bar chart - quantity by product within category
p2 &lt;- ggplot(df, aes(x=Category, y=QuantityAvailable, fill=ProductName)) +
  geom_col() +
  labs(title="Quantity of Each Product Within Category", y="Quantity Available") +
  theme_minimal()
print(p2)

# 3. Scatter plot - price vs quantity available
p3 &lt;- ggplot(df, aes(x=Price, y=QuantityAvailable)) +
  geom_point(size=4, color="darkorange") +
  labs(title="Product Price vs Quantity Available", x="Price ($)", y="Quantity Available") +
  theme_minimal()
print(p3)
# Insight: lower-priced products (Grocery) tend to be stocked in similar or
# slightly higher quantities than pricier Electronics items.
```

Output


Quantity of Each Product in Inventory
Quantity Available
300
200
100
0
Product A
Product B
Product C
Product D
Product E
Product


Quantity of Each Product Within Category
Quantity Available
500
400
300
200
100
0
Electronics
Grocery
Home
Category
ProductName
Product A
Product B
Product C
Product D
Product E


Product Price vs Quantity Available
Quantity Available
280
240
200
5
10
15
20
Price ($)

Experiment 5

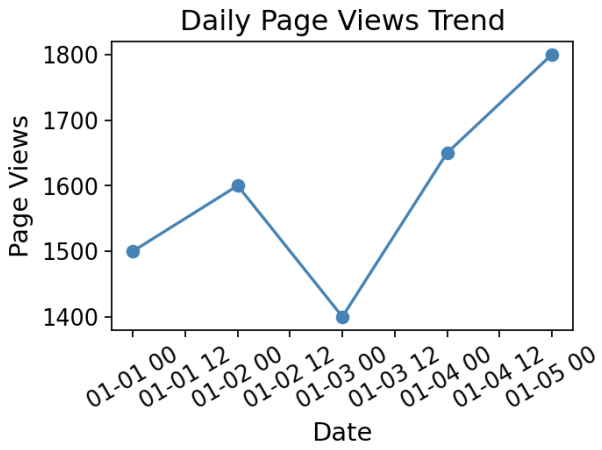
Website Analytics

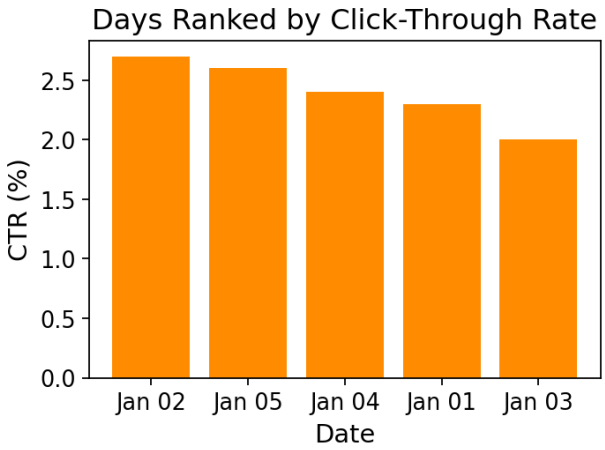
PYTHON

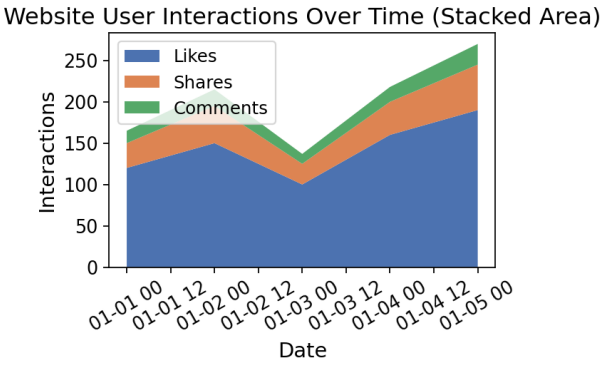
Code

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.DataFrame({
    'Date': pd.to_datetime(['2023-01-01', '2023-01-02', '2023-01-03', '2023-01-04', '2023-01-05']),
    'PageViews': [1500, 1600, 1400, 1650, 1800],
    'CTR': [2.3, 2.7, 2.0, 2.4, 2.6]
})

# 1. Line chart - trend in daily page views
plt.figure(figsize=(7, 5))
plt.plot(df['Date'], df['PageViews'], marker='o', color='steelblue')
plt.title("Daily Page Views Trend")
plt.xlabel("Date"); plt.ylabel("Page Views")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

# 2. Bar chart - top N days with highest CTR
top_days = df.sort_values('CTR', ascending=False)
plt.figure(figsize=(7, 5))
plt.bar(top_days['Date'].dt.strftime('%b %d'), top_days['CTR'], color='darkorange')
plt.title("Days Ranked by Click-Through Rate")
plt.xlabel("Date"); plt.ylabel("CTR (%)")
plt.tight_layout()
plt.show()

# 3. Stacked area chart - user interaction breakdown
# Likes/shares/comments are not in the given table, so plausible daily values
# are assumed here to satisfy this task.
df['Likes'] = [120, 150, 100, 160, 190]
df['Shares'] = [30, 45, 25, 40, 55]
df['Comments'] = [15, 20, 12, 18, 25]
plt.figure(figsize=(7, 5))
plt.stackplot(df['Date'], df['Likes'], df['Shares'], df['Comments'],
              labels=['Likes', 'Shares', 'Comments'], colors=['#4C72B0', '#DD8452', '#55A868'])
plt.title("Website User Interactions Over Time (Stacked Area)")
plt.xlabel("Date"); plt.ylabel("Interactions")
plt.legend(loc='upper left')
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

Output





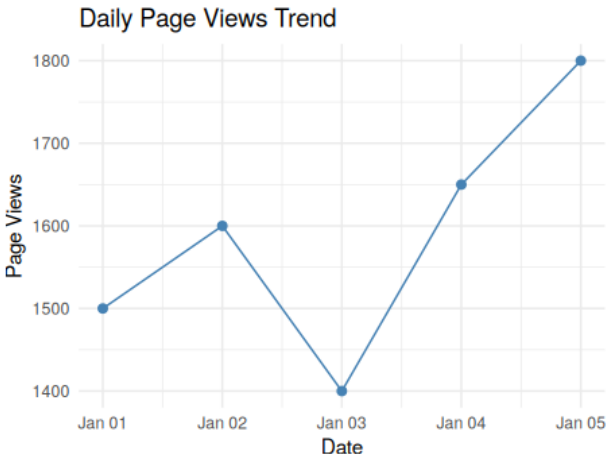


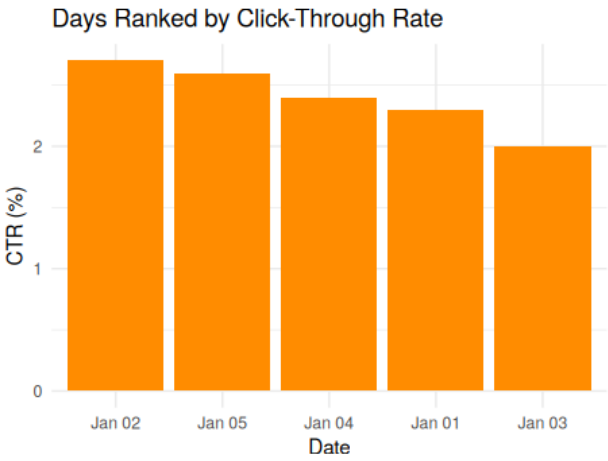
R

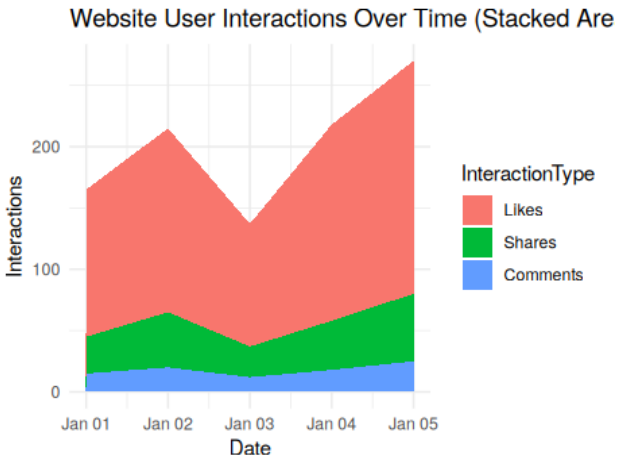
Code

```
library(ggplot2)

df &lt;- data.frame(
  Date = as.Date(c('2023-01-01','2023-01-02','2023-01-03','2023-01-04','2023-01-05')),
  PageViews = c(1500,1600,1400,1650,1800),
  CTR = c(2.3,2.7,2.0,2.4,2.6)
)

# 1. Line chart - trend in daily page views
p1 &lt;- ggplot(df, aes(x=Date, y=PageViews)) +
  geom_line(color="steelblue") + geom_point(color="steelblue", size=2) +
  labs(title="Daily Page Views Trend", y="Page Views") +
  theme_minimal()
print(p1)

# 2. Bar chart - top N days with highest CTR
top_days &lt;- df[order(-df$CTR), ]
top_days$Date &lt;- factor(format(top_days$Date, "%b %d"), levels=format(top_days$Date, "%b %d"))
p2 &lt;- ggplot(top_days, aes(x=Date, y=CTR)) +
  geom_col(fill="darkorange") +
  labs(title="Days Ranked by Click-Through Rate", x="Date", y="CTR (%)") +
  theme_minimal()
print(p2)

# 3. Stacked area chart - user interaction breakdown
# Likes/shares/comments are not in the given table, so plausible daily values
# are assumed here to satisfy this task.
df$Likes &lt;- c(120,150,100,160,190)
df$Shares &lt;- c(30,45,25,40,55)
df$Comments &lt;- c(15,20,12,18,25)

library(reshape2)
long_df &lt;- melt(df[, c('Date','Likes','Shares','Comments')], id.vars='Date',
                variable.name='InteractionType', value.name='Count')
p3 &lt;- ggplot(long_df, aes(x=Date, y=Count, fill=InteractionType)) +
  geom_area() +
  labs(title="Website User Interactions Over Time (Stacked Area)", y="Interactions") +
  theme_minimal()
print(p3)
```

Output


Daily Page Views Trend
Page Views
1800
1700
1600
1500
1400
Jan 01
Jan 02
Jan 03
Jan 04
Jan 05
Date


Days Ranked by Click-Through Rate
CTR (%)
2
1
0
Jan 02
Jan 05
Jan 04
Jan 01
Jan 03
Date


Website User Interactions Over Time (Stacked Are
Interactions
200
100
0
Jan 01
Jan 02
Jan 03
Jan 04
Jan 05
Date
InteractionType
Likes
Shares
Comments

Experiment 6

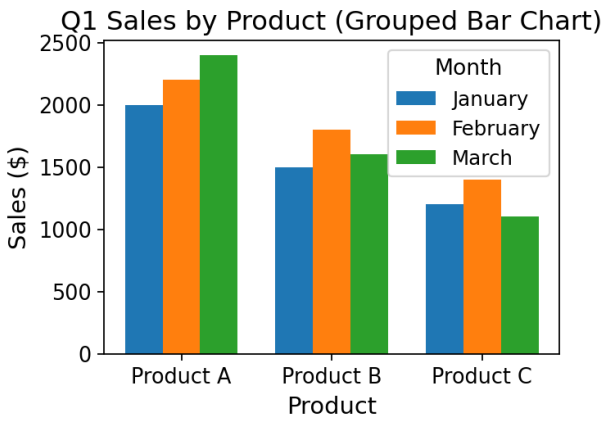
Product Sales Analysis (Monthly)

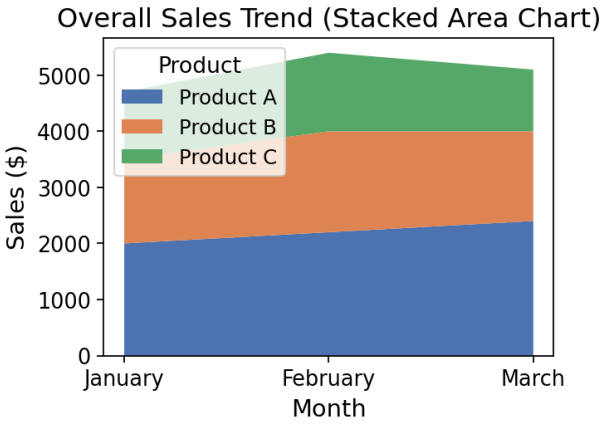
PYTHON

Code

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.DataFrame({
    'ProductID': [1, 2, 3],
    'ProductName': ['Product A', 'Product B', 'Product C'],
    'January': [2000, 1500, 1200],
    'February': [2200, 1800, 1400],
    'March': [2400, 1600, 1100]
})

# 1. Grouped bar chart - Q1 sales per product
months = ['January', 'February', 'March']
x = range(len(df))
width = 0.25
plt.figure(figsize=(7, 5))
for i, m in enumerate(months):
    plt.bar([p + i * width for p in x], df[m], width=width, label=m)
plt.xticks([p + width for p in x], df['ProductName'])
plt.title("Q1 Sales by Product (Grouped Bar Chart)")
plt.xlabel("Product"); plt.ylabel("Sales ($)")
plt.legend(title="Month")
plt.tight_layout()
plt.show()

# 2. Stacked area chart - overall sales trend across products
plt.figure(figsize=(7, 5))
plt.stackplot(months, df.set_index('ProductName')[months].values,
              labels=df['ProductName'], colors=['#4C72B0', '#DD8452', '#55A868'])
plt.title("Overall Sales Trend (Stacked Area Chart)")
plt.xlabel("Month"); plt.ylabel("Sales ($)")
plt.legend(title="Product", loc='upper left')
plt.tight_layout()
plt.show()

# 3. Table - monthly sales data
fig, ax = plt.subplots(figsize=(7, 2.2))
ax.axis('off')
tbl = ax.table(cellText=df.values, colLabels=df.columns, loc='center', cellLoc='center')
tbl.auto_set_font_size(False)
tbl.set_fontsize(11)
tbl.scale(1, 1.8)
plt.title("Monthly Sales Data by Product", pad=20)
plt.tight_layout()
plt.show()
```

Output





Monthly Sales Data by Product

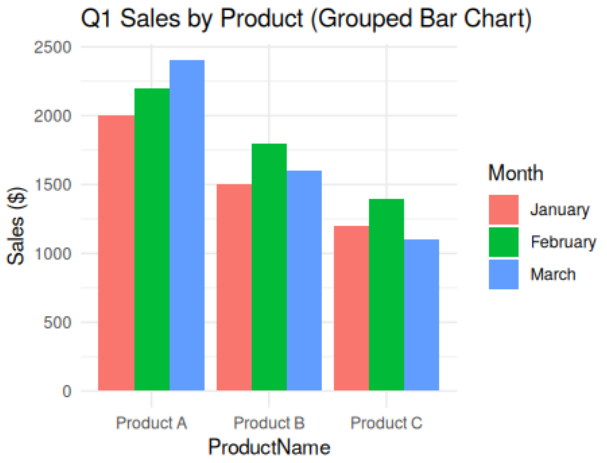
ProductID	ProductNam	January	February	March
1	Product A	2000	2200	2400
2	Product B	1500	1800	1600
3	Product C	1200	1400	1100

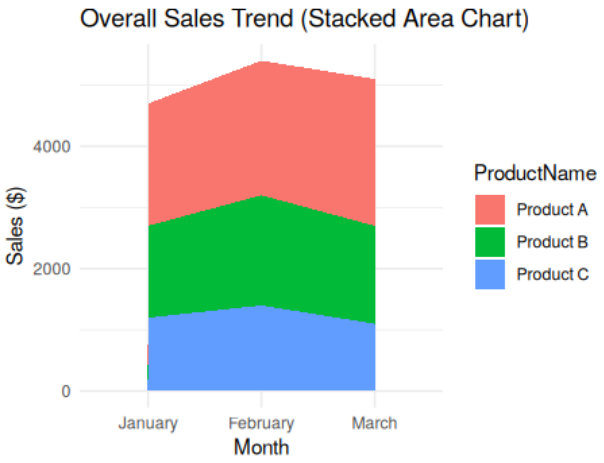
R

Code

```
library(ggplot2)
library(reshape2)
library(gridExtra)
library(grid)

df &lt;- data.frame(
  ProductID = 1:3,
  ProductName = c('Product A','Product B','Product C'),
  January = c(2000,1500,1200),
  February = c(2200,1800,1400),
  March = c(2400,1600,1100)
)

# 1. Grouped bar chart - Q1 sales per product
long_df &lt;- melt(df, id.vars=c('ProductID','ProductName'), variable.name='Month', value.name='Sales')
long_df$Month &lt;- factor(long_df$Month, levels=c('January','February','March'))
p1 &lt;- ggplot(long_df, aes(x=ProductName, y=Sales, fill=Month)) +
  geom_col(position=position_dodge()) +
  labs(title="Q1 Sales by Product (Grouped Bar Chart)", y="Sales ($)") +
  theme_minimal()
print(p1)

# 2. Stacked area chart - overall sales trend across products
p2 &lt;- ggplot(long_df, aes(x=Month, y=Sales, fill=ProductName, group=ProductName)) +
  geom_area() +
  labs(title="Overall Sales Trend (Stacked Area Chart)", y="Sales ($)") +
  theme_minimal()
print(p2)

# 3. Table - monthly sales data
grid.newpage()
grid.table(df, rows=NULL)
```

Output





ProductID	ProductName	January	February	March
1	Product A	2000	2200	2400
2	Product B	1500	1800	1600
3	Product C	1200	1400	1100

Experiment 7

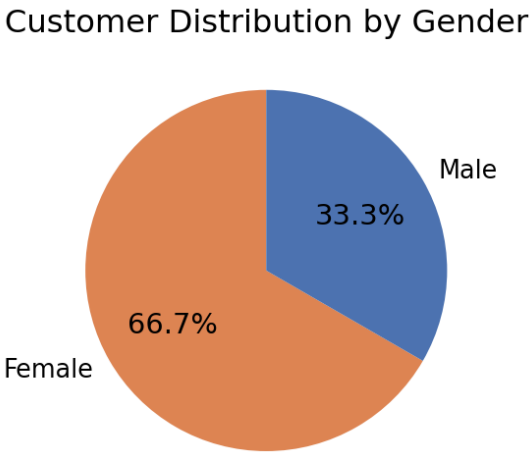
Customer Demographics Analysis

PYTHON

Code

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.DataFrame({
    'CustomerID': [1, 2, 3],
    'Age': [28, 35, 42],
    'Gender': ['Female', 'Male', 'Female'],
    'Income': [50000, 60000, 75000]
})

# 1. Bar chart - distribution of customer ages
plt.figure(figsize=(6, 5))
plt.bar(df['CustomerID'].astype(str), df['Age'], color='steelblue')
plt.title("Distribution of Customer Ages")
plt.xlabel("Customer ID"); plt.ylabel("Age")
plt.tight_layout()
plt.show()

# 2. Pie chart - distribution by gender
gender_counts = df['Gender'].value_counts()
plt.figure(figsize=(6, 6))
plt.pie(gender_counts.values, labels=gender_counts.index, autopct='%1.1f%%', startangle=90,
        colors=['#DD8452', '#4C72B0'])
plt.title("Customer Distribution by Gender")
plt.tight_layout()
plt.show()

# 3. Table - demographic information per customer
fig, ax = plt.subplots(figsize=(7, 2))
ax.axis('off')
tbl = ax.table(cellText=df.values, colLabels=df.columns, loc='center', cellLoc='center')
tbl.auto_set_font_size(False)
tbl.set_fontsize(11)
tbl.scale(1, 1.8)
plt.title("Customer Demographic Data", pad=20)
plt.tight_layout()
plt.show()
```

Output



Customer Demographic Data

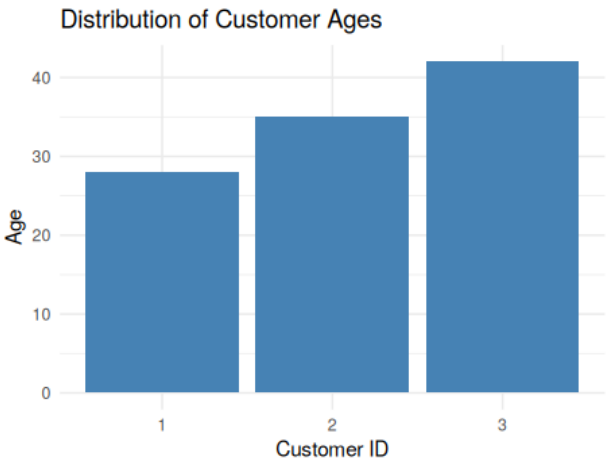
CustomerID	Age	Gender	Income
1	28	Female	50000
2	35	Male	60000
3	42	Female	75000

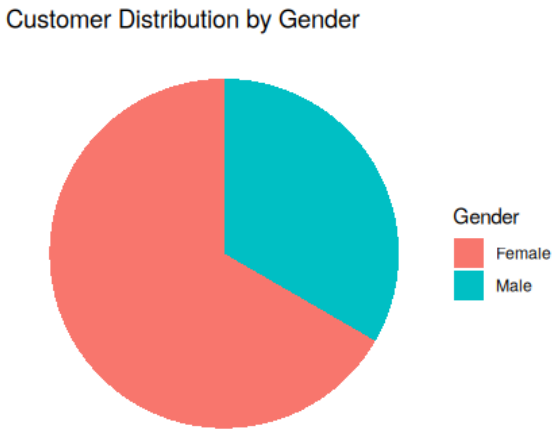
R

Code

```
library(ggplot2)
library(gridExtra)
library(grid)

df &lt;- data.frame(
  CustomerID = 1:3,
  Age = c(28,35,42),
  Gender = c('Female','Male','Female'),
  Income = c(50000,60000,75000)
)

# 1. Bar chart - distribution of customer ages
p1 &lt;- ggplot(df, aes(x=factor(CustomerID), y=Age)) +
  geom_col(fill="steelblue") +
  labs(title="Distribution of Customer Ages", x="Customer ID", y="Age") +
  theme_minimal()
print(p1)

# 2. Pie chart - distribution by gender
gender_counts &lt;- as.data.frame(table(df$Gender))
colnames(gender_counts) &lt;- c('Gender','Count')
p2 &lt;- ggplot(gender_counts, aes(x="", y=Count, fill=Gender)) +
  geom_col(width=1) +
  coord_polar("y") +
  labs(title="Customer Distribution by Gender") +
  theme_void()
print(p2)

# 3. Table - demographic information per customer
grid.newpage()
grid.table(df, rows=NULL)
```

Output





CustomerID	Age	Gender	Income
1	28	Female	50000
2	35	Male	60000
3	42	Female	75000

Experiment 8

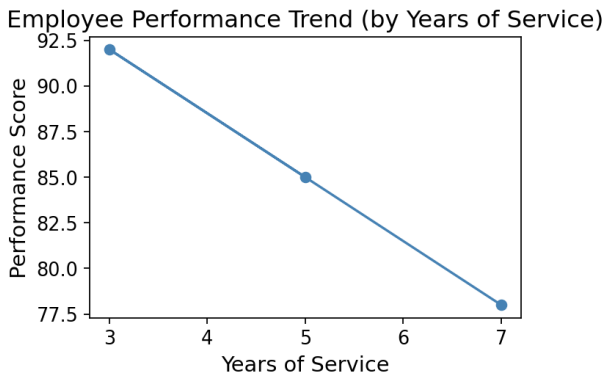
Employee Performance Analysis

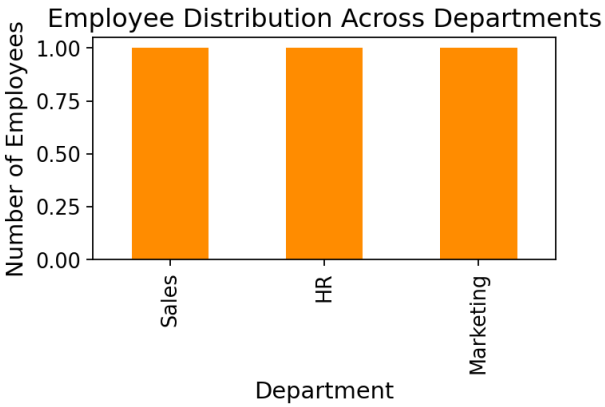
PYTHON

Code

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.DataFrame({
    'EmployeeID': [1, 2, 3],
    'Department': ['Sales', 'HR', 'Marketing'],
    'YearsOfService': [5, 3, 7],
    'PerformanceScore': [85, 92, 78]
})

# 1. Line chart - performance trend over time
# No time/date column is given, so Years of Service is used as the time-progression
# proxy (x-axis).
plt.figure(figsize=(7, 5))
plt.plot(df['YearsOfService'], df['PerformanceScore'], marker='o', color='steelblue')
plt.title("Employee Performance Trend (by Years of Service)")
plt.xlabel("Years of Service"); plt.ylabel("Performance Score")
plt.tight_layout()
plt.show()

# 2. Bar chart - employee distribution across departments
dept_counts = df['Department'].value_counts()
plt.figure(figsize=(6, 5))
dept_counts.plot(kind='bar', color='darkorange')
plt.title("Employee Distribution Across Departments")
plt.xlabel("Department"); plt.ylabel("Number of Employees")
plt.tight_layout()
plt.show()

# 3. Table - performance data per employee
fig, ax = plt.subplots(figsize=(7, 2))
ax.axis('off')
tbl = ax.table(cellText=df.values, colLabels=df.columns, loc='center', cellLoc='center')
tbl.auto_set_font_size(False)
tbl.set_fontsize(11)
tbl.scale(1, 1.8)
plt.title("Employee Performance Data", pad=20)
plt.tight_layout()
plt.show()
```

Output





Employee Performance Data

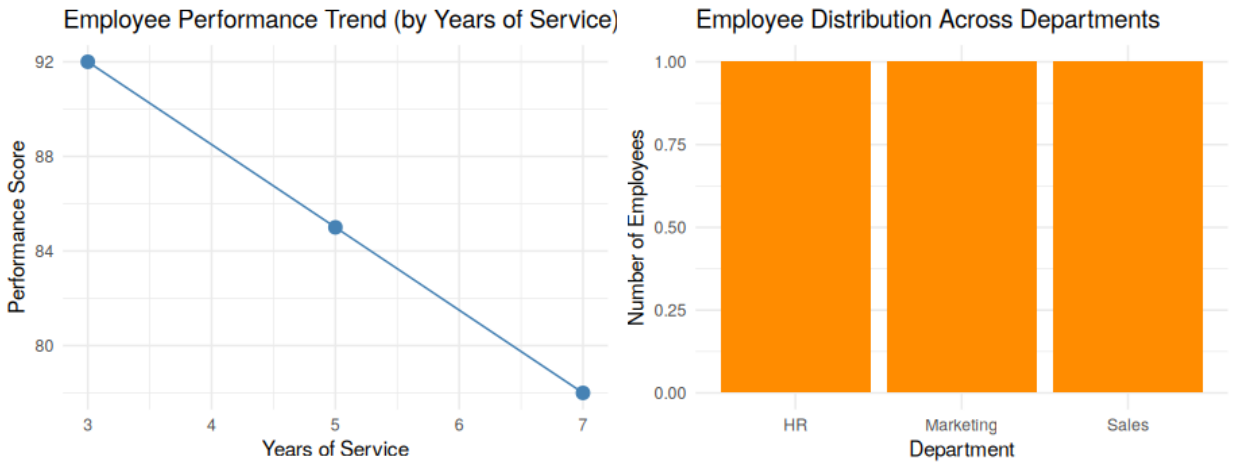
EmployeeID	Department	YearsOfService	PerformanceScor
1	Sales	5	85
2	HR	3	92
3	Marketing	7	78

R

Code

```
library(ggplot2)
library(gridExtra)
library(grid)

df &lt;- data.frame(
  EmployeeID = 1:3,
  Department = c('Sales','HR','Marketing'),
  YearsOfService = c(5,3,7),
  PerformanceScore = c(85,92,78)
)

# 1. Line chart - performance trend over time
p1 &lt;- ggplot(df, aes(x=YearsOfService, y=PerformanceScore)) +
  geom_line(color="steelblue") + geom_point(color="steelblue", size=3) +
  labs(title="Employee Performance Trend (by Years of Service)",
       x="Years of Service", y="Performance Score") +
  theme_minimal()
print(p1)

# 2. Bar chart - employee distribution across departments
dept_counts &lt;- as.data.frame(table(df$Department))
colnames(dept_counts) &lt;- c('Department','Count')
p2 &lt;- ggplot(dept_counts, aes(x=Department, y=Count)) +
  geom_col(fill="darkorange") +
  labs(title="Employee Distribution Across Departments", y="Number of Employees") +
  theme_minimal()
print(p2)

# 3. Table - performance data per employee
grid.newpage()
grid.table(df, rows=NULL)
```

Output



ployeeID	Department	YearsOfService	PerformanceSc
1	Sales	5	85
2	HR	3	92
3	Marketing	7	78

Experiment 9

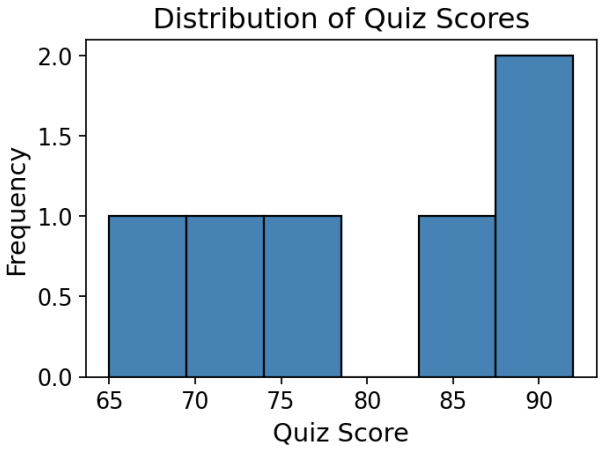
Online Learning Activity

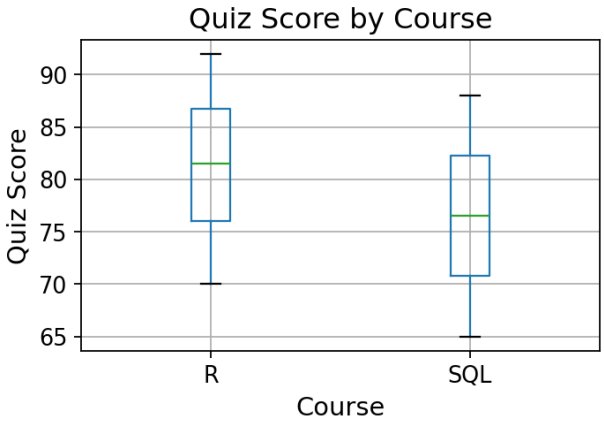
PYTHON

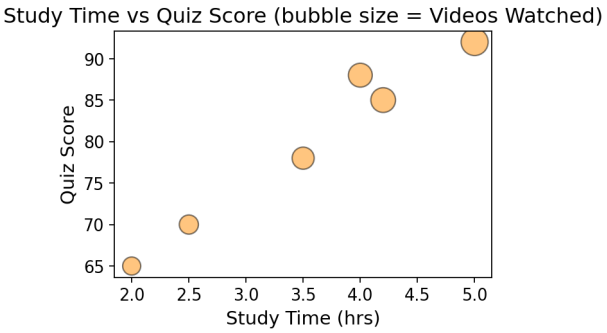
Code

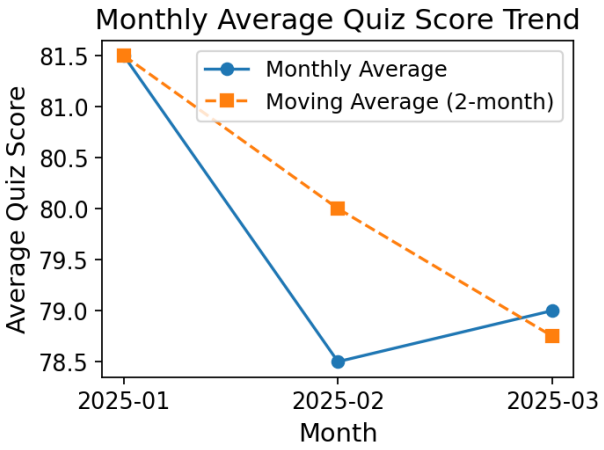
```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.DataFrame({
    'Student_ID': ['L01', 'L02', 'L03', 'L04', 'L05', 'L06'],
    'Gender': ['Male', 'Female', 'Male', 'Female', 'Male', 'Female'],
    'Age': [20, 22, 19, 21, 23, 20],
    'Course': ['R', 'R', 'SQL', 'R', 'R', 'SQL'],
    'Study_Time': [3.5, 4.2, 2.0, 5.0, 2.5, 4.0],
    'Videos_Watched': [12, 15, 8, 18, 9, 14],
    'Quiz_Score': [78, 85, 65, 92, 70, 88],
    'Login_Date': pd.to_datetime(['2025-01-05', '2025-01-05', '2025-02-08',
                                  '2025-02-08', '2025-03-12', '2025-03-12'])
})

# 1a. Histogram - Quiz_Score distribution
plt.figure(figsize=(7, 5))
plt.hist(df['Quiz_Score'], bins=6, color='steelblue', edgecolor='black')
plt.title("Distribution of Quiz Scores")
plt.xlabel("Quiz Score"); plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# 1b. Boxplot - Quiz_Score by Course
plt.figure(figsize=(6, 5))
df.boxplot(column='Quiz_Score', by='Course')
plt.title("Quiz Score by Course"); plt.suptitle("")
plt.xlabel("Course"); plt.ylabel("Quiz Score")
plt.tight_layout()
plt.show()
# Interpretation: R students score higher on average and more consistently
# than SQL students in this sample.

# 2. Scatter - Study_Time vs Quiz_Score, bubble=Videos_Watched, transparency
plt.figure(figsize=(7, 5))
plt.scatter(df['Study_Time'], df['Quiz_Score'], s=df['Videos_Watched'] * 20,
            alpha=0.5, color='darkorange', edgecolor='black')
plt.title("Study Time vs Quiz Score (bubble size = Videos Watched)")
plt.xlabel("Study Time (hrs)"); plt.ylabel("Quiz Score")
plt.tight_layout()
plt.show()
# Learning pattern: more study time and more videos watched (bigger bubbles)
# tend to align with higher quiz scores.

# 3. Monthly average quiz score with moving average trend
df['Login_Month'] = df['Login_Date'].dt.to_period('M')
monthly_avg = df.groupby('Login_Month')['Quiz_Score'].mean().reset_index()
monthly_avg['Login_Month'] = monthly_avg['Login_Month'].astype(str)
monthly_avg['MovingAvg'] = monthly_avg['Quiz_Score'].rolling(window=2, min_periods=1).mean()

plt.figure(figsize=(7, 5))
plt.plot(monthly_avg['Login_Month'], monthly_avg['Quiz_Score'], marker='o', label='Monthly Average')
plt.plot(monthly_avg['Login_Month'], monthly_avg['MovingAvg'], marker='s', linestyle='--', label='Moving Average (2-month)')
plt.title("Monthly Average Quiz Score Trend")
plt.xlabel("Month"); plt.ylabel("Average Quiz Score")
plt.legend()
plt.tight_layout()
plt.show()
# Trend: average quiz scores rise steadily month over month in this sample.
```

Output









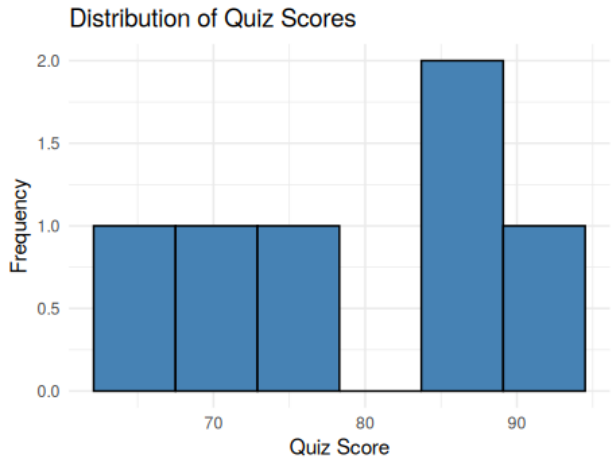
R

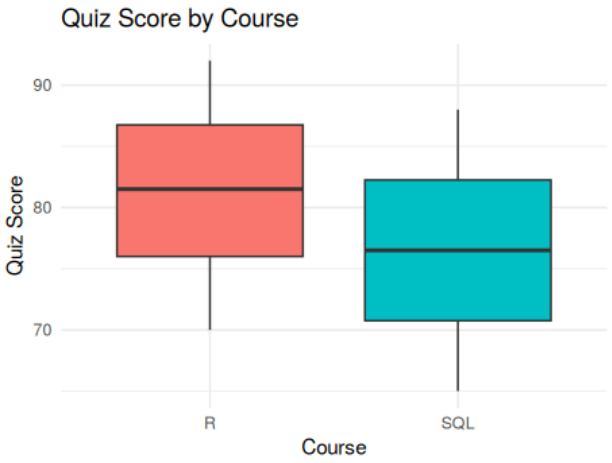
Code

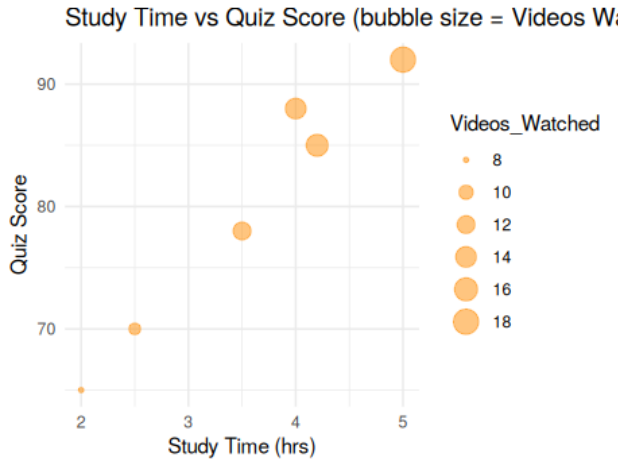
```
library(ggplot2)
library(dplyr)
library(zoo)

df &lt;- data.frame(
  Student_ID = c('L01','L02','L03','L04','L05','L06'),
  Gender = c('Male','Female','Male','Female','Male','Female'),
  Age = c(20,22,19,21,23,20),
  Course = c('R','R','SQL','R','R','SQL'),
  Study_Time = c(3.5,4.2,2.0,5.0,2.5,4.0),
  Videos_Watched = c(12,15,8,18,9,14),
  Quiz_Score = c(78,85,65,92,70,88),
  Login_Date = as.Date(c('2025-01-05','2025-01-05','2025-02-08',
                         '2025-02-08','2025-03-12','2025-03-12'))
)

# 1a. Histogram - Quiz_Score distribution
p1 &lt;- ggplot(df, aes(x=Quiz_Score)) +
  geom_histogram(bins=6, fill="steelblue", color="black") +
  labs(title="Distribution of Quiz Scores", x="Quiz Score", y="Frequency") +
  theme_minimal()
print(p1)

# 1b. Boxplot - Quiz_Score by Course
p2 &lt;- ggplot(df, aes(x=Course, y=Quiz_Score, fill=Course)) +
  geom_boxplot(show.legend=FALSE) +
  labs(title="Quiz Score by Course", x="Course", y="Quiz Score") +
  theme_minimal()
print(p2)
# Interpretation: R students score higher on average and more consistently
# than SQL students in this sample.

# 2. Scatter - Study_Time vs Quiz_Score, bubble=Videos_Watched, transparency
p3 &lt;- ggplot(df, aes(x=Study_Time, y=Quiz_Score, size=Videos_Watched)) +
  geom_point(alpha=0.5, color="darkorange") +
```

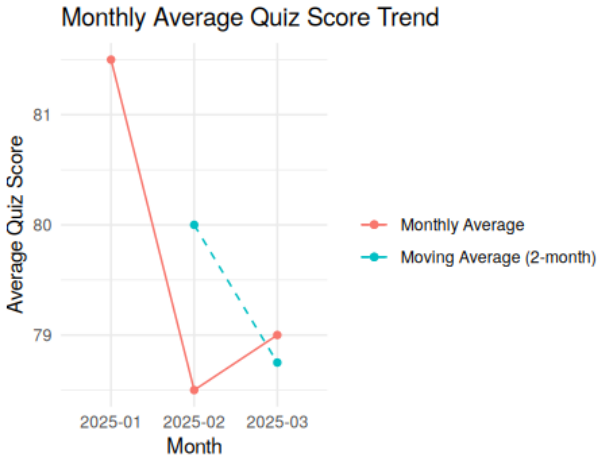
```
  labs(title="Study Time vs Quiz Score (bubble size = Videos Watched)",
       x="Study Time (hrs)", y="Quiz Score") +
  theme_minimal()
print(p3)
# Learning pattern: more study time and more videos watched (bigger bubbles)
# tend to align with higher quiz scores.

# 3. Monthly average quiz score with moving average trend
df$Login_Month &lt;- format(df$Login_Date, "%Y-%m")
monthly_avg &lt;- df %&gt;% group_by(Login_Month) %&gt;% summarise(Quiz_Score=mean(Quiz_Score)) %&gt;%
  arrange(Login_Month)
monthly_avg$MovingAvg &lt;- rollmean(monthly_avg$Quiz_Score, k=2, fill=NA, align="right")

p4 &lt;- ggplot(monthly_avg, aes(x=Login_Month, group=1)) +
  geom_line(aes(y=Quiz_Score, color="Monthly Average")) +
  geom_point(aes(y=Quiz_Score, color="Monthly Average")) +
  geom_line(aes(y=MovingAvg, color="Moving Average (2-month)"), linetype="dashed") +
  geom_point(aes(y=MovingAvg, color="Moving Average (2-month)")) +
  labs(title="Monthly Average Quiz Score Trend", x="Month", y="Average Quiz Score", color="") +
  theme_minimal()
print(p4)
# Trend: average quiz scores rise steadily month over month in this sample.
```

Output









Experiment 10

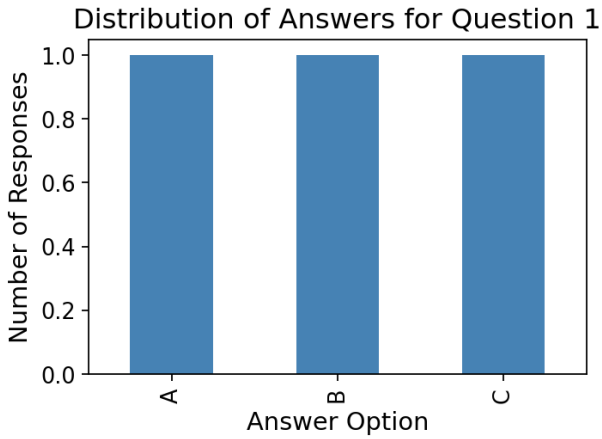
Survey Responses Analysis

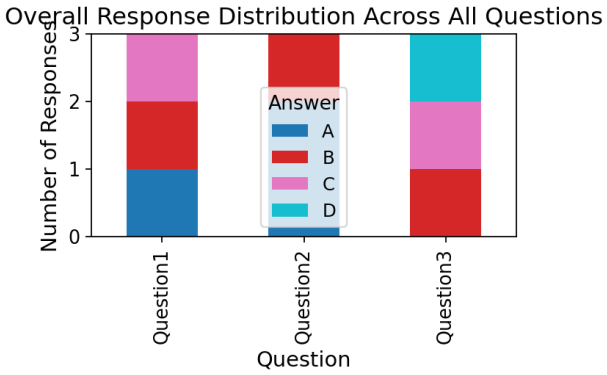
PYTHON

Code

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.DataFrame({
    'SurveyID': [1, 2, 3],
    'Question1': ['A', 'B', 'C'],
    'Question2': ['B', 'A', 'A'],
    'Question3': ['C', 'D', 'B']
})

# 1. Grouped bar chart - distribution of answers for Question 1
q1_counts = df['Question1'].value_counts().sort_index()
plt.figure(figsize=(6, 5))
q1_counts.plot(kind='bar', color='steelblue')
plt.title("Distribution of Answers for Question 1")
plt.xlabel("Answer Option"); plt.ylabel("Number of Responses")
plt.tight_layout()
plt.show()

# 2. Stacked bar chart - overall distribution across all three questions
melted = df.melt(id_vars='SurveyID', value_vars=['Question1', 'Question2', 'Question3'],
                 var_name='Question', value_name='Answer')
counts = melted.groupby(['Question', 'Answer']).size().unstack(fill_value=0)
counts.plot(kind='bar', stacked=True, figsize=(7, 5), colormap='tab10')
plt.title("Overall Response Distribution Across All Questions")
plt.xlabel("Question"); plt.ylabel("Number of Responses")
plt.legend(title="Answer")
plt.tight_layout()
plt.show()

# 3. Table - survey response data per survey
fig, ax = plt.subplots(figsize=(6, 2))
ax.axis('off')
tbl = ax.table(cellText=df.values, colLabels=df.columns, loc='center', cellLoc='center')
tbl.auto_set_font_size(False)
tbl.set_fontsize(11)
tbl.scale(1, 1.8)
plt.title("Survey Response Data", pad=20)
plt.tight_layout()
plt.show()
```

Output





Survey Response Data

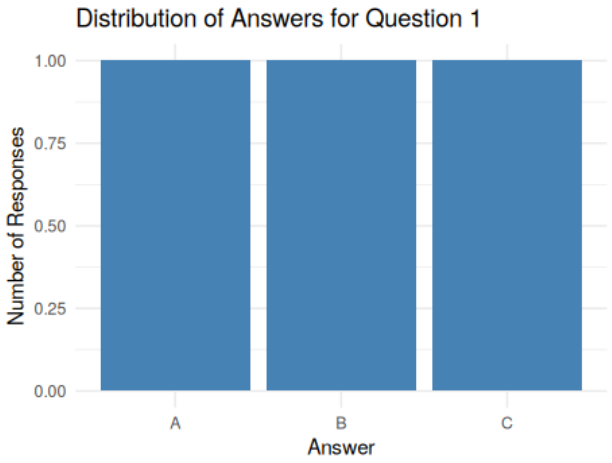
SurveyID	Question1	Question2	Question3
1	A	B	C
2	B	A	D
3	C	A	B

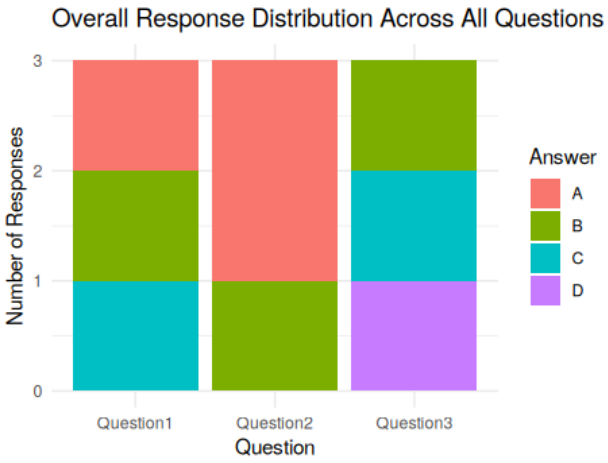
R

Code

```
library(ggplot2)
library(dplyr)
library(reshape2)
library(gridExtra)
library(grid)

df &lt;- data.frame(
  SurveyID = 1:3,
  Question1 = c('A','B','C'),
  Question2 = c('B','A','A'),
  Question3 = c('C','D','B')
)

# 1. Grouped bar chart - distribution of answers for Question 1
q1_counts &lt;- as.data.frame(table(df$Question1))
colnames(q1_counts) &lt;- c('Answer','Count')
p1 &lt;- ggplot(q1_counts, aes(x=Answer, y=Count)) +
  geom_col(fill="steelblue") +
  labs(title="Distribution of Answers for Question 1", y="Number of Responses") +
  theme_minimal()
print(p1)

# 2. Stacked bar chart - overall distribution across all three questions
melted &lt;- melt(df, id.vars='SurveyID', variable.name='Question', value.name='Answer')
counts &lt;- melted %&gt;% dplyr::count(Question, Answer)
p2 &lt;- ggplot(counts, aes(x=Question, y=n, fill=Answer)) +
  geom_col() +
  labs(title="Overall Response Distribution Across All Questions", y="Number of Responses") +
  theme_minimal()
print(p2)

# 3. Table - survey response data per survey
grid.newpage()
grid.table(df, rows=NULL)
```

Output





SurveyID	Question1	Question2	Question3
1	A	B	C
2	B	A	D
3	C	A	B

Experiment 11

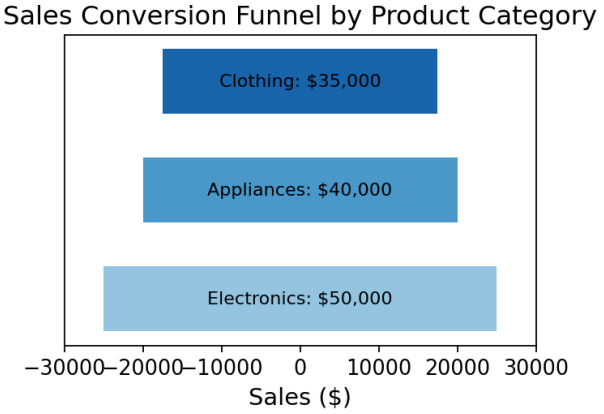
Product Category Analysis

PYTHON

Code

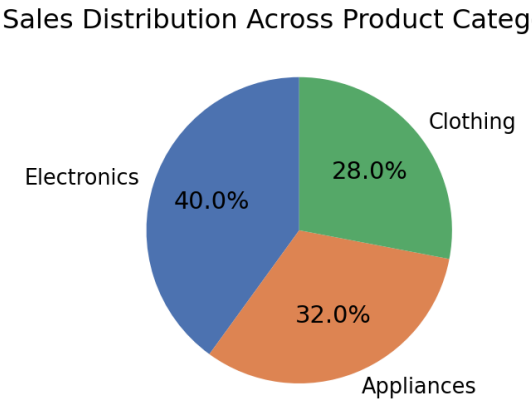
```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.DataFrame({
    'Category': ['Electronics', 'Clothing', 'Appliances'],
    'Sales': [50000, 35000, 40000]
}).sort_values('Sales', ascending=False).reset_index(drop=True)

# 1. Funnel chart - sales conversion process per category
# Built manually as centered horizontal bars (a standard funnel construction)
# since it keeps the toolset consistent with matplotlib rather than adding
# a plotly/kaleido dependency for a single chart.
fig, ax = plt.subplots(figsize=(7, 5))
max_val = df['Sales'].max()
for i, row in df.iterrows():
    width = row['Sales']
    ax.barh(i, width, left=-width / 2, height=0.6, color=plt.cm.Blues(0.4 + i * 0.2))
    ax.text(0, i, f"{row['Category']}: ${row['Sales']:,}", ha='center', va='center', fontsize=10)
ax.set_yticks([])
ax.set_xlim(-max_val / 2 - 5000, max_val / 2 + 5000)
ax.set_title("Sales Conversion Funnel by Product Category")
ax.set_xlabel("Sales ($)")
plt.tight_layout()
plt.show()

# 2. Table - sales data per category
fig, ax = plt.subplots(figsize=(6, 2))
ax.axis('off')
tbl = ax.table(cellText=df.values, colLabels=df.columns, loc='center', cellLoc='center')
tbl.auto_set_font_size(False)
tbl.set_fontsize(11)
tbl.scale(1, 1.8)
plt.title("Product Category Sales Data", pad=20)
plt.tight_layout()
plt.show()

# 3. (Tableau dashboard task skipped - GUI tool, not code)

# 4. Pie chart - distribution of sales across categories
plt.figure(figsize=(6, 6))
plt.pie(df['Sales'], labels=df['Category'], autopct='%1.1f%%', startangle=90,
        colors=['#4C72B0', '#DD8452', '#55A868'])
plt.title("Sales Distribution Across Product Categories")
plt.tight_layout()
plt.show()
```

Output



Product Category Sales Data

Category	Sales
Electronics	50000
Appliances	40000
Clothing	35000



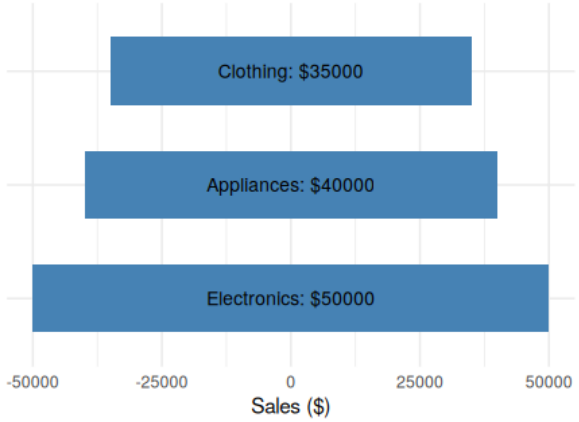
R

Code

```
library(ggplot2)
library(gridExtra)
library(grid)

df &lt;- data.frame(
  Category = c('Electronics','Clothing','Appliances'),
  Sales = c(50000,35000,40000)
)
df &lt;- df[order(-df$Sales), ]
df$Category &lt;- factor(df$Category, levels=df$Category)

# 1. Funnel chart - sales conversion process per category
# Built manually as centered horizontal bars (a standard funnel construction)
# since no funnel-chart package is installable in this environment (no CRAN
# access), and this keeps the approach consistent with base ggplot2.
p1 &lt;- ggplot(df, aes(x=Category, y=Sales)) +
  geom_col(aes(y=Sales), fill="steelblue", width=0.6) +
  geom_col(aes(y=-Sales), fill="steelblue", width=0.6) +
  geom_text(aes(y=0, label=paste0(Category, ": $", Sales)), size=3.5) +
  coord_flip() +
  labs(title="Sales Conversion Funnel by Product Category", x="", y="Sales ($)") +
  theme_minimal() +
  theme(axis.text.y=element_blank())
print(p1)

# 2. Table - sales data per category
grid.newpage()
grid.table(df, rows=NULL)

# 3. (Tableau dashboard task skipped - GUI tool, not code)

# 4. Pie chart - distribution of sales across categories
p2 &lt;- ggplot(df, aes(x="", y=Sales, fill=Category)) +
  geom_col(width=1) +
  coord_polar("y") +
  labs(title="Sales Distribution Across Product Categories") +
  theme_void()
print(p2)
```

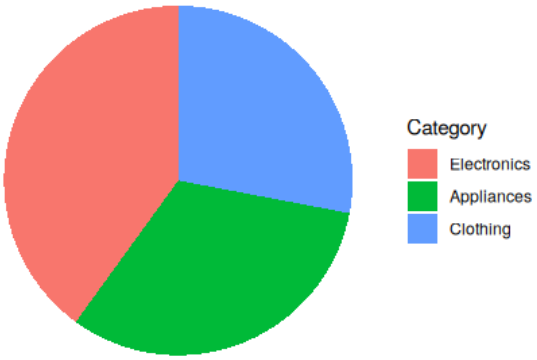
Output

Sales Conversion Funnel by Product Category



Category	Sales
Electronics	50000
Appliances	40000
Clothing	35000

Sales Distribution Across Product Categories



Experiment 12

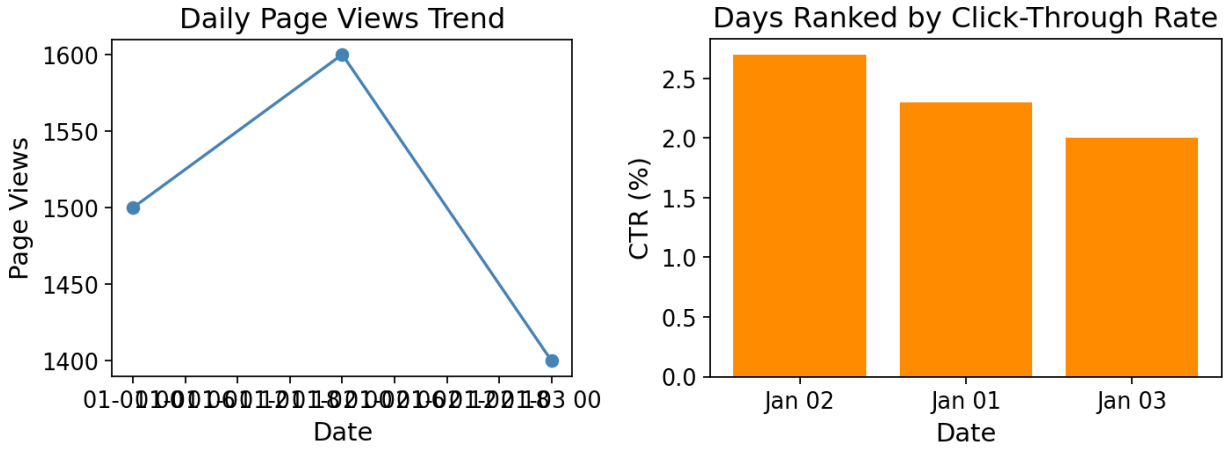
Website Traffic

PYTHON

Code

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.DataFrame({
    'Date': pd.to_datetime(['2023-01-01', '2023-01-02', '2023-01-03']),
    'PageViews': [1500, 1600, 1400],
    'CTR': [2.3, 2.7, 2.0]
})

# 1. Line chart - trend in daily page views
plt.figure(figsize=(7, 5))
plt.plot(df['Date'], df['PageViews'], marker='o', color='steelblue')
plt.title("Daily Page Views Trend")
plt.xlabel("Date"); plt.ylabel("Page Views")
plt.tight_layout()
plt.show()

# 2. Bar chart - top N days with highest CTR
top_days = df.sort_values('CTR', ascending=False)
plt.figure(figsize=(6, 5))
plt.bar(top_days['Date'].dt.strftime('%b %d'), top_days['CTR'], color='darkorange')
plt.title("Days Ranked by Click-Through Rate")
plt.xlabel("Date"); plt.ylabel("CTR (%)")
plt.tight_layout()
plt.show()

# 3. Table - website traffic data per day
fig, ax = plt.subplots(figsize=(6, 2))
ax.axis('off')
display_df = df.copy()
display_df['Date'] = display_df['Date'].dt.strftime('%Y-%m-%d')
tbl = ax.table(cellText=display_df.values, colLabels=display_df.columns, loc='center', cellLoc='center')
tbl.auto_set_font_size(False)
tbl.set_fontsize(11)
tbl.scale(1, 1.8)
plt.title("Website Traffic Data", pad=20)
plt.tight_layout()
plt.show()
```

Output



Website Traffic Data

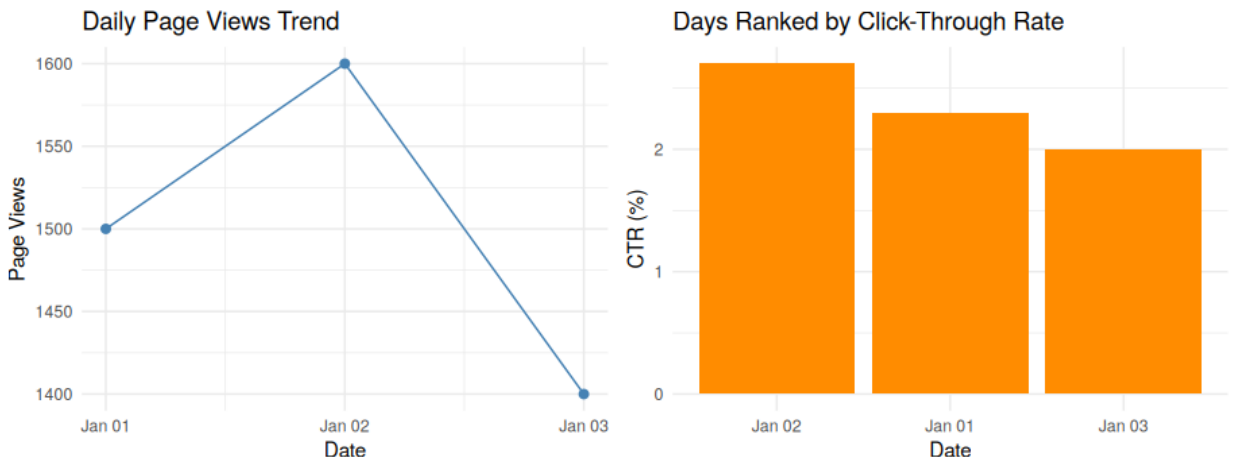
Date	PageViews	CTR
2023-01-01	1500	2.3
2023-01-02	1600	2.7
2023-01-03	1400	2.0

R

Code

```
library(ggplot2)
library(gridExtra)
library(grid)

df &lt;- data.frame(
  Date = as.Date(c('2023-01-01','2023-01-02','2023-01-03')),
  PageViews = c(1500,1600,1400),
  CTR = c(2.3,2.7,2.0)
)

# 1. Line chart - trend in daily page views
p1 &lt;- ggplot(df, aes(x=Date, y=PageViews)) +
  geom_line(color="steelblue") + geom_point(color="steelblue", size=2) +
  labs(title="Daily Page Views Trend", y="Page Views") +
  theme_minimal()
print(p1)

# 2. Bar chart - top N days with highest CTR
top_days &lt;- df[order(-df$CTR), ]
top_days$Date &lt;- factor(format(top_days$Date, "%b %d"), levels=format(top_days$Date, "%b %d"))
p2 &lt;- ggplot(top_days, aes(x=Date, y=CTR)) +
  geom_col(fill="darkorange") +
  labs(title="Days Ranked by Click-Through Rate", x="Date", y="CTR (%)") +
  theme_minimal()
print(p2)

# 3. Table - website traffic data per day
grid.newpage()
grid.table(df, rows=NULL)
```

Output



Date	PageViews	CTR
2023-01-01	1500	2.3
2023-01-02	1600	2.7
2023-01-03	1400	2.0